TravelSecure

Travel Insurance Management System

Software Requirements Specification (SRS)

Document Version:	1.0.0
Release Date:	March 10, 2026
Document Type:	Software Requirements Specification
Standard:	IEEE 830 / IEEE 29148

1. Introduction

1.1 Purpose

This document defines the complete software requirements for the TravelSecure Travel Insurance Management System. It serves as the primary technical agreement between stakeholders, architects, and developers. The document covers functional requirements, system architecture, data models, security constraints, and non-functional requirements to ensure all parties share a clear and unambiguous understanding of what the system must do and how it must behave.

1.2 Scope

TravelSecure is a web-based travel insurance management platform that digitises and streamlines the end-to-end process of purchasing, managing, and claiming travel insurance. The system is built on a decoupled architecture with an Angular frontend and a .NET 8 Clean Architecture backend, secured throughout via JWT-based stateless authentication. It supports four distinct user roles – Administrator, Customer, Agent, and Claim Officer – each with dedicated workflows and access boundaries. The system encompasses:

- Identity and Access Management via stateless JWT authentication and role-based access control.
- Dynamic premium calculation based on traveler age, trip duration, and destination risk.
- Multi-role policy workflows covering plan browsing, policy requests, Agent review, and customer renewal.
- A structured, manual claim review process where Claim Officers evaluate evidence and record settlement decisions.
- Real-time analytics and performance dashboards for Agents, Claim Officers, and Administrators.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token – a compact, URL-safe means of representing claims between parties
SPA	Single Page Application
ORM	Object-Relational Mapper (Entity Framework Core)
API	Application Programming Interface (RESTful HTTP)
EF Core	Entity Framework Core – the ORM used for database operations
DTO	Data Transfer Object – a shaped payload class for API communication
CORS	Cross-Origin Resource Sharing
RBAC	Role-Based Access Control
RFC 7807	HTTP API Problem Details standard for error responses

1.4 References

- Angular Documentation – https://angular.io/docs (v21+)
- Microsoft .NET 8 Documentation – https://learn.microsoft.com/en-us/dotnet/
- Entity Framework Core Docs – https://learn.microsoft.com/en-us/ef/core/
- JSON Web Tokens – https://jwt.io/introduction
- RFC 7807 – Problem Details for HTTP APIs
- TailwindCSS v3 Documentation – https://tailwindcss.com/docs

- Chart.js v4 Documentation – https://www.chartjs.org/docs/latest/
- SendGrid Official Documentation – https://docs.sendgrid.com/

1.5 Document Overview

This document is organized as follows:

- Section 1 – Introduction, scope, and definitions
- Section 2 – Overall system description and constraints
- Section 3 – Functional requirements, organized by user role
- Section 4 – Non-functional requirements
- Section 5 – System architecture and design constraints
- Section 6 – External interface requirements
- Section 7 – Data requirements and entity model
- Section 8 – Security requirements
- Section 9 – System design diagrams
- Section 10 – Appendices

2. Overall System Description

2.1 Product Perspective

TravelSecure is a purpose-built web application for the travel insurance domain. It replaces manual, paper-based insurance workflows with a structured, role-driven digital platform. The system consists of an Angular SPA frontend and a .NET 8 Clean Architecture backend connected via a RESTful API. Claim evidence documents are stored in the backend server's local uploads folder. Email notifications are dispatched via SendGrid.

2.2 System Context

The system interacts with the following actors and external services:

- External Users: Customers seeking travel insurance coverage via a self-service web portal.
- Internal Users: Agents and Claim Officers processing policy requests and reviewing claim submissions.
- SendGrid: External SMTP service used to dispatch email notifications across all user roles.

2.3 High-Level Product Functions

- User Provisioning: Self-service customer registration and Admin-controlled provisioning of Agents and Claim Officers.
- Policy Lifecycle: From plan browsing and premium preview through policy request, Agent review, payment, and renewal.
- Claim Lifecycle: Structured manual review process – Customer submits a claim with evidence → Admin assigns to Claim Officer → Officer reviews documents and approves or rejects → Settlement recorded.
- Premium Calculation: Formula-driven calculation based on traveler age, trip duration, and destination risk multiplier configured by the Admin.
- Analytics & Reporting: Role-specific dashboards showing sales performance for Agents, claim resolution metrics for Officers, and financial trends for Administrators.

2.4 User Classes and Characteristics

User Role	Characteristics
Administrator	Internal power users with full system access. Responsible for plan configuration, user provisioning, assignment of policies and claims, and monitoring financial analytics.
Customer	End users who register via the public portal to browse plans, purchase travel insurance, manage their active policies, and raise claims. Expected to have no technical background; the interface must be straightforward and self-explanatory.
Agent	Internal sales staff who review and approve or reject policy requests assigned to them by the Admin. They monitor their own sales performance via a dedicated dashboard.
Claim Officer	Internal domain experts who manually review claim submissions, examine uploaded evidence documents, and record approval or rejection decisions along with the final settlement amount.

2.5 Operating Environment

- Frontend: Angular SPA (v21+) served via modern browsers (Chrome, Firefox, Edge, Safari).
- Backend: ASP.NET Core 8 Web API, deployable via Docker or direct IIS/Kestrel hosting.
- Database: Microsoft SQL Server (LocalDB for development; full SQL Server for production).
- File Storage: Claim evidence documents are stored in the backend server's local uploads folder; no external storage service is used.
- Email: SendGrid SMTP API used for transactional notifications across all user roles.

2.6 Design & Implementation Constraints

- Clean Architecture must be strictly maintained – no direct database calls are permitted from the API/Presentation layer; all data access must flow through the Application layer via repository interfaces.
- JWT tokens are the sole authentication mechanism; no session-based or cookie authentication is permitted anywhere in the system.
- All role-specific data queries must extract user identity from the JWT claim (userId); identity must never be sourced from the request body or URL parameters.
- Database schema changes must be managed exclusively through EF Core migrations; manual schema edits to the database are prohibited.
- The Angular frontend must be modularised with lazy-loaded feature modules per role (Admin, Customer, Agent, Claim Officer) to enforce separation of concerns.
- All inter-layer communication in the backend must use DTOs; domain entities must never be directly exposed through API responses.
- All API error responses must conform to the RFC 7807 Problem Details format via the global exception handling middleware.
- The frontend must be fully responsive, supporting tablet (768px+) and desktop (1280px+) viewports using TailwindCSS v3.
- Password storage must use BCrypt or Argon2 hashing; plaintext or weakly hashed passwords are strictly prohibited.

3. Functional Requirements

Functional requirements are organised by user role to reflect the role-based access model of TravelSecure. Each module lists the capabilities available to its primary actor(s).

3.1 Authentication & Profile (All Roles)

ID	Requirement Name	Description
FR-AUTH-001	Secure User Login	Users must provide a valid email and password. The system returns a signed JWT upon successful authentication.
FR-AUTH-002	Customer Self-Registration	Public users can register as a Customer. All PII fields are validated via server-side Regex rules.
FR-AUTH-003	JWT Refresh Mechanism	The system supports configurable JWT expiry (default 60 min) with a sliding-window refresh to maintain stateless sessions.
FR-AUTH-004	Admin User Provisioning	Administrators can create, edit, and deactivate accounts for Agents and Claim Officers.
FR-AUTH-005	Secure Password Storage	All passwords are hashed using BCrypt or Argon2 before persistence. Plaintext storage is forbidden.

3.2 Administrator – Plan & System Management

ID	Requirement Name	Description
FR-ADM-001	Create Insurance Plan	Admins can define a plan with name, type (Single Trip / Multi-Trip / Student), and maximum coverage amount.
FR-ADM-002	Define Coverage Types	Admins can attach multiple coverage categories (Medical, Baggage, Trip Cancellation, etc.) with individual limits per plan.
FR-ADM-003	Configure Premium Logic	Admins can set base prices and per-day rates used by the premium calculation formula.
FR-ADM-004	Destination Risk Management	Admins can add, update, or remove destination risk entries and assign multipliers (1.0x – 3.0x) to specific countries.
FR-ADM-005	Assign Agent to Policy	Admins can assign a pending policy request to a specific Agent for review and approval.
FR-ADM-006	Assign Claim to Officer	Admins can assign unprocessed claims to specific Claim Officers for manual review.
FR-ADM-007	User Account Management	Admins can activate or deactivate user accounts (Agents, Claim Officers, Customers) and view categorised user lists.
FR-ADM-008	Growth Analytics	Admins can view monthly premium generation trends, all policies filtered by status, and system-wide financial KPIs.

3.3 Customer – Policy & Claim Self-Service

ID	Requirement Name	Description
FR-CUS-001	Browse & Filter Plans	Customers can browse all active insurance plans, filter by coverage type, and view detailed plan metadata.
FR-CUS-002	Premium Preview	Customers can enter traveler age, number of travel days, and destination to receive a real-time premium estimate before purchase.
FR-CUS-003	Policy Request	Customers can submit a policy request by selecting a plan and providing traveler and nominee details.
FR-CUS-004	Policy Purchase	Customers can finalise payment for an Agent-approved policy, transitioning its status to Active.
FR-CUS-005	Invoice Download	Customers can download a dynamically generated PDF invoice for each successful payment.
FR-CUS-006	Policy Renewal	Customers can renew an existing active policy by specifying additional days; the pro-rated premium is recalculated and collected.
FR-CUS-007	View Policies	Customers can view all their policies – Active, Expired, PaymentPending, and Interested – and see coverage breakdowns per policy.
FR-CUS-008	Raise a Claim	Customers can file a claim against an active policy by providing claim details and uploading supporting evidence documents (PDF, JPG, PNG).
FR-CUS-009	Track Claim Status	Customers can view the full history and current status of all their submitted claims.

3.4 Agent – Policy Validation & Sales Pipeline

ID	Requirement Name	Description
FR-AGT-001	View Assigned Policies	Agents can view all policy requests currently assigned to them by an Administrator.
FR-AGT-002	Approve Policy	Agents can review applicant and traveler details and approve a policy request, transitioning its status to PaymentPending.
FR-AGT-003	View Sold Portfolio	Agents can view a portfolio of all policies they have approved that are now Active.
FR-AGT-004	Sales Dashboard	Agents can view a visual dashboard comparing their total Policies Sold vs. Policies Assigned.

3.5 Claim Officer – Claim Review & Settlement

ID	Requirement Name	Description
FR-OFF-001	View Assigned Claims	Claim Officers can view all claims assigned to them by the Administrator for manual review.
FR-OFF-002	Review Claim Evidence	Claim Officers can access and view uploaded evidence documents (PDFs and images) stored in the backend uploads folder.
FR-OFF-003	Coverage Validation	The system automatically enforces that a claim amount does not exceed the policy's coverage limit for the relevant category.
FR-OFF-004	Approve / Reject Claim	Claim Officers can approve or reject a claim with mandatory written review notes (10–500 characters).
FR-OFF-005	Settlement	Claim Officers specify the final settled amount (must be ≤ requested claim amount) and mark the claim as Settled.
FR-OFF-006	Performance Metrics	Claim Officers can view personal performance metrics including approval rates and total settlement amounts.

4. Non-Functional Requirements

4.1 Performance

ID	Requirement	Target Metric
NFR-PERF-01	API Response Time	95th-percentile response time for all API endpoints must be under 500ms under normal load.
NFR-PERF-02	Premium Calculation	Real-time premium preview must return a result within 200ms.
NFR-PERF-03	Concurrent Users	System must support a minimum of 500 concurrent active sessions without degradation.

4.2 Reliability & Availability

ID	Requirement	Target Metric
NFR-REL-01	System Availability	99.9% uptime guaranteed during standard business hours.
NFR-REL-02	Data Persistence	Zero data loss on any committed transaction; all writes must be atomic.
NFR-REL-03	Error Recovery	The system must gracefully recover from unhandled exceptions via global middleware without data corruption.

4.3 Scalability

ID	Requirement	Description
NFR-SCL-01	Horizontal Scaling	Stateless JWT authentication enables multiple Web API instances behind a load balancer without session affinity.
NFR-SCL-02	Database Scalability	The repository pattern abstracts the data store, enabling migration to a NoSQL engine for high-volume log data if required.

4.4 Maintainability & Testability

ID	Requirement	Description
NFR-MNT-01	Unit Test Coverage	Minimum 80% code coverage for the Application layer measured by automated test runs in CI.
NFR-MNT-02	Dependency Injection	All services and repositories must be registered via IoC container to enable easy mocking and stubbing.
NFR-MNT-03	Swagger Docs	All API endpoints must be documented in the auto-generated Swagger / OpenAPI specification.

4.5 Usability

ID	Requirement	Description
NFR-USE-01	Responsive Design	UI must be fully functional on tablet (768px+) and desktop (1280px+) viewports using TailwindCSS adaptive layout.
NFR-USE-02	Error Feedback	All validation failures must surface user-friendly messages compliant with RFC 7807 Problem Details format.

5. System Architecture & Design Constraints

5.1 Architectural Overview

TravelSecure follows the Clean Architecture (Onion Architecture) pattern, ensuring that core business logic remains fully independent of external frameworks, databases, and UI implementations. All dependency arrows point inward toward the Domain layer.

5.2 Frontend Architecture (Angular SPA)

The frontend is a highly modular Angular v21+ Single Page Application structured as:

- Feature Modules: Lazy-loaded modules per role – Admin, Customer, Agent, Claim Officer.
- Core Module: Singleton services including AuthService, HTTP Interceptors, and Route Guards.
- Shared Module: Reusable UI components – Toasts, Cards, Layout wrappers, and Chart widgets.
- State Management: Reactive data flows implemented with RxJS Observables and Subjects.
- Styling: TailwindCSS v3 utility-first CSS with responsive breakpoints.
- Charts: Chart.js v4 for analytics dashboards.

5.3 Backend Architecture (.NET 8 Clean Architecture)

The backend is structured into four distinct, dependency-isolated layers:

5.3.1 Domain Layer (Core)

- Contains all Domain Entities, Value Objects, and Domain-specific Exceptions.
- Has zero dependencies on any other layer or external framework.
- Defines the canonical business reality of the insurance domain.

5.3.2 Application Layer

- Declares interfaces for all services and repositories (Dependency Inversion Principle).
- Implements all core use cases via Service classes that orchestrate domain rules.
- Contains DTOs, AutoMapper profiles, and validation logic.

5.3.3 Infrastructure Layer

- Implements EF Core repository classes and the DbContext with MS SQL Server.
- Manages all EF Core Migrations and database seeding.
- Provides concrete implementations for SendGrid email integration, local file upload handling, and Payment Gateway clients.

5.3.4 Presentation Layer (Web API)

- ASP.NET Core 8 Controllers exposing RESTful endpoints documented via Swagger.
- Global Exception Handling Middleware that logs, sanitises, and formats errors as RFC 7807 APIResponse<T>.
- JWT Bearer Authentication Middleware with CORS policy enforcement.

5.4 Deployment Architecture

- Containerisation: Docker multi-stage build produces a lean production image.
- Database: MS SQL Server (LocalDB for development; full SQL Server for production).
- File Storage: Claim evidence documents are stored in the backend server's local uploads folder.
- CI/CD: Automated pipelines trigger build, test, and deploy on each merge to main.

6. External Interface Requirements

6.1 User Interface

- The frontend is an Angular SPA rendered in modern web browsers (Chrome 120+, Firefox 120+, Edge 120+, Safari 17+).
- Layouts are built with TailwindCSS v3 using CSS Grid and Flexbox, supporting tablet (768px+) and desktop (1280px+) viewports.
- All user-facing validation errors are displayed inline using RFC 7807-compliant messages.

6.2 API Interface

All backend functionality is exposed via a RESTful HTTP API. The full endpoint catalog is listed below, grouped by module:

6.2.1 Authentication (api/auth)

Method	Endpoint	Role	Description
POST	/api/auth/register	Anonymous	Customer self-registration with PII validation.
POST	/api/auth/admin/register	Admin	Register a new Agent or Claim Officer account.
POST	/api/auth/login	Anonymous	User login; returns a signed JWT on success. Supports reCAPTCHA.
POST	/api/auth/forgot-password	Anonymous	Request a password reset token delivered via email.
POST	/api/auth/reset-password	Anonymous	Reset password using a valid token received by email.

6.2.2 Claims Management (api/claims)

Method	Endpoint	Role	Description
POST	/api/claims	Customer	Raise a new insurance claim for an active policy.
GET	/api/claims	Customer	View history of all claims submitted by the logged-in customer.
PATCH	/api/claims/{id}/assign-officer	Admin	Assign a specific Claim Officer to a pending claim.
GET	/api/claims/unassigned	Admin	List all claims that have not yet been assigned to an officer.
GET	/api/claims/assigned	Admin	List all claims currently assigned to any officer.
GET	/api/claims/all	Admin	Comprehensive view of all claims in the system.
PATCH	/api/claims/{claimId}/review	ClaimOfficer	Approve or reject a claim with mandatory review notes.
PATCH	/api/claims/{claimId}/settle	ClaimOfficer	Finalise and settle a claim with a specified settlement amount.
GET	/api/claims/officer/assigned	ClaimOfficer	View all claims assigned specifically to the logged-in officer.
GET	/api/claims/officer/performance	ClaimOfficer	Get personal performance metrics (approval rates, settlement totals).

6.2.3 Destination Risk Management (api/destination-risk)

Method	Endpoint	Role	Description
GET	/api/destination-risk	Admin, Customer	Get list of all countries and their risk multipliers.

Method	Endpoint	Role	Description
POST	/api/destination-risk	Admin	Add a new destination risk entry.
PUT	/api/destination-risk/{id}	Admin	Update an existing destination's risk level or multiplier.
DELETE	/api/destination-risk/{id}	Admin	Remove a destination from the risk database.

6.2.4 File Uploads (api/upload)

Method	Endpoint	Role	Description
POST	/api/upload	Customer	Securely upload claim documentation (PDF, JPG, PNG) to the backend uploads folder.

6.2.5 Insurance Plans (api/plans)

Method	Endpoint	Role	Description
POST	/api/plans	Admin	Create a new insurance plan.
GET	/api/plans	Any	List all active insurance plans.
PUT	/api/plans/{id}	Admin	Update plan details or coverage limits.
PATCH	/api/plans/{id}/activate	Admin	Enable a previously deactivated plan.
PATCH	/api/plans/{id}/deactivate	Admin	Disable a plan from public view.
GET	/api/plans/browse	Customer	Filter and browse plans by specific coverage types.
GET	/api/plans/{id}	Any	Get detailed metadata for a specific plan.

6.2.6 Policy Management (api/policies)

Method	Endpoint	Role	Description
POST	/api/policies	Customer	Submit a request to purchase a policy (initial status: Interested).
GET	/api/policies/active	Customer	List all currently active policies for the logged-in customer.
GET	/api/policies/payment-pending	Customer	List policies approved by an Agent but awaiting payment.
GET	/api/policies/my-policies	Customer	Full history of all policies (Active, Expired, Pending, Interested).
GET	/api/policies/{id}/coverages	Customer	Fetch the coverage breakdown for a specific policy's plan.
PATCH	/api/policies/{policyId}/renew	Customer	Renew or extend an existing active policy.
POST	/api/policies/{policyId}/buy	Customer	Finalise payment and transition the policy status to Active.
PATCH	/api/policies/{policyId}/approve	Agent	Validate and approve a policy request.
GET	/api/policies/agent/pending	Agent	List policy requests assigned to the agent awaiting approval.

Method	Endpoint	Role	Description
GET	/api/policies/agent/sold	Agent	Portfolio of policies approved and now active by this agent.
PATCH	/api/policies/{id}/assign-agent	Admin	Manually assign an Agent to a policy request.
GET	/api/policies/admin	Admin	Filter and view all policies in the system by status.
GET	/api/policies/{id}	Any	Retrieve full details of a specific policy by ID.

6.2.7 Premium Calculation (api/premium)

Method	Endpoint	Role	Description
POST	/api/premium/calculate	Any	Calculate dynamic premium based on traveler age, number of travel days, and destination risk multiplier.

6.2.8 User Management (api/users)

Method	Endpoint	Role	Description
GET	/api/users/profile	Any	Retrieve profile details for the currently logged-in user.
GET	/api/users/agents	Admin	List all registered Agents.
GET	/api/users/customers	Admin	List all registered Customers.
GET	/api/users/claim-officers	Admin	List all registered Claim Officers.
PATCH	/api/users/{id}/activate	Admin	Reactivate a suspended user account.
PATCH	/api/users/{id}/deactivate	Admin	Suspend a user account.

6.3 Email Notification Interface

- Outbound emails are sent via the SendGrid API using server-side API key authentication.
- Notifications are triggered on: policy approval, payment confirmation, claim status change, and settlement.
- Email templates are HTML-formatted and personalised per recipient role.

6.4 File Upload Interface

- Claim evidence documents are uploaded via POST /api/upload and stored in the backend server's local uploads folder.
- Accepted formats: .pdf, .jpg, and .png only. Maximum file size is 5MB per document.
- The stored file path is persisted in the ClaimDocument entity and served to Claim Officers during manual review.

7. Data Requirements & Entity Model

The domain model consists of nine entities derived from the system's ER diagram. All entities are persisted in Microsoft SQL Server and managed exclusively through EF Core migrations.

7.1 Entity Definitions

7.1.1 Users

Central identity entity shared by all roles. Role field acts as a discriminator to determine access boundaries.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
Name	nvarchar	NOT NULL	Full display name of the user.
Email	nvarchar	UNIQUE	Login identifier; validated via Regex on registration.
PasswordHash	nvarchar	NOT NULL	BCrypt/Argon2 hashed password; plaintext never stored.
Role	nvarchar	NOT NULL	Discriminator: Admin | Customer | Agent | ClaimOfficer.
IsActive	bit	NOT NULL	Soft-delete flag; inactive users cannot log in.
CreatedAt	datetime	NOT NULL	Timestamp of account creation.
DateOfBirth	date	nullable	Used for age-based premium multiplier calculation.
AadhaarNumber	nvarchar	nullable	Government ID for identity verification (Customer).
PhoneNumber	nvarchar	nullable	Contact number.
Address	nvarchar	nullable	Residential address.
DateOfJoin	datetime	nullable	Date the Agent or Officer joined the organisation.
CommissionRate	decimal	nullable	Agent-specific commission rate on sold policies.
ResetToken	nvarchar	nullable	One-time password reset token; sent via email.
ResetTokenExpiry	datetime	nullable	Expiry timestamp for the reset token.

7.1.2 InsurancePlans

Master catalogue of travel insurance products defined and managed by the Administrator.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
PolicyName	nvarchar	NOT NULL	Descriptive plan name (e.g., Bronze, Silver, Gold).
PlanType	nvarchar	NOT NULL	Category: Single Trip | Multi-Trip | Student.
MaxCoverageAmount	decimal	NOT NULL	Upper limit of total coverage provided by this plan.
IsActive	bit	NOT NULL	Controls whether the plan is visible for customer browsing.

7.1.3 Coverages

Each InsurancePlan has one or more coverage line items defining per-category benefit limits.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
CoverageType	nvarchar	NOT NULL	Category name, e.g., Medical, Baggage, Trip Cancellation.
CoverageAmount	decimal	NOT NULL	Maximum claimable amount for this coverage category.
InsurancePlanId	int	FK	FK → InsurancePlans. Links coverage to its parent plan.

7.1.4 PremiumRules

Stores the age-bracket multipliers and per-day rate configured by the Admin for each plan. Used by the premium calculation engine.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
InsurancePlanId	int	FK	FK → InsurancePlans.
BasePrice	decimal	NOT NULL	Fixed base cost of the plan before multipliers.
AgeBelow30Multiplier	decimal	NOT NULL	Premium multiplier applied when traveler age is below 30.
AgeBetween30And50Multiplier	decimal	NOT NULL	Premium multiplier applied when traveler age is between 30 and 50.
AgeAbove50Multiplier	decimal	NOT NULL	Premium multiplier applied when traveler age is above 50.
PerDayRate	decimal	NOT NULL	Per-day rate multiplied by trip duration to calculate total premium.

7.1.5 DestinationRisks

Lookup table of destination countries with associated risk multipliers that adjust the final premium amount.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
Destination	nvarchar	NOT NULL	Country or destination name.
RiskMultiplier	decimal	NOT NULL	Risk factor (1.0x – 3.0x) applied on top of the base premium.

7.1.6 Policies

Core transactional entity representing a customer's insurance policy. Captures snapshot fields at time of purchase to preserve historical accuracy even if plan rules change later.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
CustomerId	int	FK	FK → Users (Customer who purchased the policy).
AgentId	int	FK	FK → Users (Agent assigned by Admin for approval).
InsurancePlanId	int	FK	FK → InsurancePlans.
DestinationCountry	nvarchar	NOT NULL	Travel destination selected at time of purchase.
StartDate	datetime	NOT NULL	Coverage commencement date.
EndDate	datetime	NOT NULL	Coverage expiry date; updated on renewal.
PremiumAmount	decimal	NOT NULL	Calculated premium amount paid by the customer.
Status	nvarchar	NOT NULL	Interested | PaymentPending | Active | Expired | Cancelled.
CoverageId	int	nullable	Reference to the primary coverage selected.
AgeMultiplier	decimal	NOT NULL	Age bracket multiplier captured at purchase time.
PurchaseAge	int	NOT NULL	Traveler's age at the time of policy purchase.
AgeAbove50Multiplier	decimal	NOT NULL	Snapshot of the above-50 multiplier at purchase time.
AgeBelow30Multiplier	decimal	NOT NULL	Snapshot of the below-30 multiplier at purchase time.
AgeBetween30And50Multiplier	decimal	NOT NULL	Snapshot of the 30–50 multiplier at purchase time.
BasePrice	decimal	NOT NULL	Snapshot of the plan's base price at purchase time.
CoveragesJson	nvarchar	nullable	JSON snapshot of all coverage details at the time of purchase.
MaxCoverageAmount	decimal	NOT NULL	Snapshot of the plan's maximum coverage at purchase time.
PerDayRate	decimal	NOT NULL	Snapshot of the per-day rate at purchase time.
PlanName	nvarchar	NOT NULL	Snapshot of the plan name at purchase time.
PlanType	nvarchar	NOT NULL	Snapshot of the plan type at purchase time.

7.1.7 Travelers

Records all travelers (primary and co-travelers) associated with a policy.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
PolicyId	int	FK	FK → Policies.
FullName	nvarchar	NOT NULL	Full name of the traveler.
DateOfBirth	date	NOT NULL	Date of birth; used for age validation (Student plan gate).
AadhaarCard	nvarchar	nullable	Government ID of the traveler for verification.
TravelerType	nvarchar	NOT NULL	Discriminator: Primary | CoTraveler.
Relationship	nvarchar	nullable	Relationship to the primary policyholder (for co-travelers).
CreatedAt	datetime	NOT NULL	Timestamp of record creation.

7.1.8 Claims

Represents a claim raised by a customer against an active policy. Progresses through a manual review workflow handled by a Claim Officer.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
PolicyId	int	FK	FK → Policies.
ClaimType	nvarchar	NOT NULL	Category of the claim (e.g., Medical, Baggage).
ClaimAmount	decimal	NOT NULL	Total amount requested by the customer.
Status	nvarchar	NOT NULL	Submitted | UnderReview | Approved | Rejected | Settled.
AssignedOfficerId	int	FK	FK → Users (Claim Officer assigned by Admin).
CreatedDate	datetime	NOT NULL	Timestamp when the claim was submitted.
SettledAmount	decimal	nullable	Final amount approved by the Officer (≤ ClaimAmount).
SettledDate	datetime	nullable	Timestamp when the claim was settled.
Description	nvarchar	NOT NULL	Customer's description of the claim incident.
ReviewNotes	nvarchar	nullable	Officer's written notes recorded during manual review (10–500 chars).

7.1.9 ClaimDocuments

Stores references to evidence files uploaded by customers in support of a claim.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
ClaimId	int	FK	FK → Claims.
Url	nvarchar	NOT NULL	Server-relative path to the uploaded file in the backend uploads folder.

7.1.10 Payments

Records premium payment transactions linked to a policy purchase or renewal.

Attribute	Type	Constraint	Description
Id	int	PK	Auto-increment primary key.
PolicyId	int	FK	FK → Policies.
Amount	decimal	NOT NULL	Total premium amount paid.
PaymentDate	datetime	NOT NULL	Timestamp of the payment transaction.
Status	nvarchar	NOT NULL	Success | Failed | Pending.

7.2 Entity Relationships

Relationship	Cardinality	Description
Users → Policies (as Customer)	1 : Many	One customer can hold multiple policies.
Users → Policies (as Agent)	1 : Many	One agent can be assigned to multiple policies.
Users → Claims (as Officer)	1 : Many	One claim officer can be assigned to multiple claims.
InsurancePlans → Coverages	1 : Many	One plan has one or more coverage line items.
InsurancePlans → PremiumRules	1 : 1	Each plan has exactly one set of premium rules.
InsurancePlans → Policies	1 : Many	One plan can be purchased as multiple policies.
DestinationRisks → Policies	1 : Many	A destination risk entry is referenced by multiple policies.
Policies → Travelers	1 : Many	One policy can have one primary traveler and multiple co-travelers.
Policies → Claims	1 : Many	One active policy can have multiple claims raised against it.
Policies → Payments	1 : Many	One policy can have multiple payment records (purchase + renewals).
Claims → ClaimDocuments	1 : Many	One claim can have multiple supporting evidence documents.

7.3 Data Validation Rules

7.3.1 Business Logic Validation

Rule ID	Description
V-BUS-001	Student Plan Age Gate: If PlanType == ‘Student’, the primary traveler’s age (derived from DateOfBirth in Travelers) must be between 15 and 30 years inclusive.
V-BUS-002	Duplicate Policy Guard: A customer may not hold more than one active policy of the same PlanType simultaneously.
V-BUS-003	Claim Ceiling: The ClaimAmount must not exceed the CoverageAmount defined for the matching CoverageType in the policy’s Coverages.

Rule ID	Description
V-BUS-004	Settlement Cap: The SettledAmount recorded by a Claim Officer must be less than or equal to the requested ClaimAmount.
V-BUS-005	Active Policy Requirement: A claim can only be raised against a policy with Status == Active.
V-BUS-006	Risk Multiplier Range: DestinationRisks.RiskMultiplier must be between 1.0 and 3.0 inclusive.

7.3.2 Technical Validation

Rule ID	Description
V-TECH-001	File Format: Only .pdf, .jpg, and .png formats are accepted for ClaimDocuments uploads.
V-TECH-002	File Size: Maximum allowable document size is 5MB per upload.
V-TECH-003	Review Notes Length: Claims.ReviewNotes must be between 10 and 500 characters.
V-TECH-004	Email Format: Users.Email is validated against RFC 5322 compliant Regex on registration and login.
V-TECH-005	Reset Token Expiry: ResetToken must be validated against ResetTokenExpiry before allowing a password reset.

8. Security Requirements

8.1 Authentication

- All protected endpoints require a valid JWT Bearer token in the Authorization header.
- JWTs are signed using HMAC-SHA256 with a 256-bit secret key stored in environment configuration.
- Token expiry is set to 60 minutes; a sliding-window refresh mechanism allows seamless session continuation without re-login.
- Tokens are stateless – the server maintains no session store.

8.2 Authorisation (RBAC)

Role	Permitted Actions
Admin	Full system configuration, user provisioning, plan management, analytics, policy & claim assignment, and refund initiation.
Agent	View and approve/reject assigned policy requests; access personal sales dashboard.
Claim Officer	View assigned claims, manually review claim evidence, record approval/rejection decisions, and enter settlement amounts.
Customer	Self-service plan browsing, policy purchase, policy renewal, and claim submission.

8.3 Password Security

- All passwords must be hashed using BCrypt (minimum cost factor 12) or Argon2id before storage.
- Plaintext, MD5, and SHA-1 hashed passwords are strictly forbidden.
- Password complexity requirements are enforced at registration via server-side Regex validation.

8.4 Data Protection

- All data in transit is encrypted via TLS 1.2 or higher (HTTPS enforced).
- Sensitive PII fields are never returned in API responses beyond the minimum necessary.
- Internal exception details (stack traces, connection strings) are stripped by the global exception handler before sending error responses to clients.

8.5 CORS Policy

- CORS is configured to allow requests only from the registered Angular SPA origin.
- Wildcard origins (*) are prohibited in production deployments.

8.6 Error & Exception Management

The system implements a Global Exception Handling Middleware pipeline:

- Log: Full error details including stack trace are recorded in the internal logging store.
- Sanitise: All internal implementation details are stripped from the outbound response to prevent information leakage.
- Format: Errors are returned to clients as a standardised APIResponse<T> object compliant with RFC 7807 Problem Details.

9. System Design Diagrams

This section presents the UML design diagrams that describe the structural and behavioural aspects of the TravelSecure system. These diagrams serve as the visual reference for implementation and are aligned with the functional and data requirements defined in Sections 3 and 7.

9.1 Use Case Diagram

The Use Case Diagram illustrates the interactions between the four system actors – Customer, Admin, Agent, and Claim Officer – and the use cases available to each. Include and extend relationships show dependent sub-flows such as adding travelers when purchasing a policy, and settling or rejecting a claim during the review process.

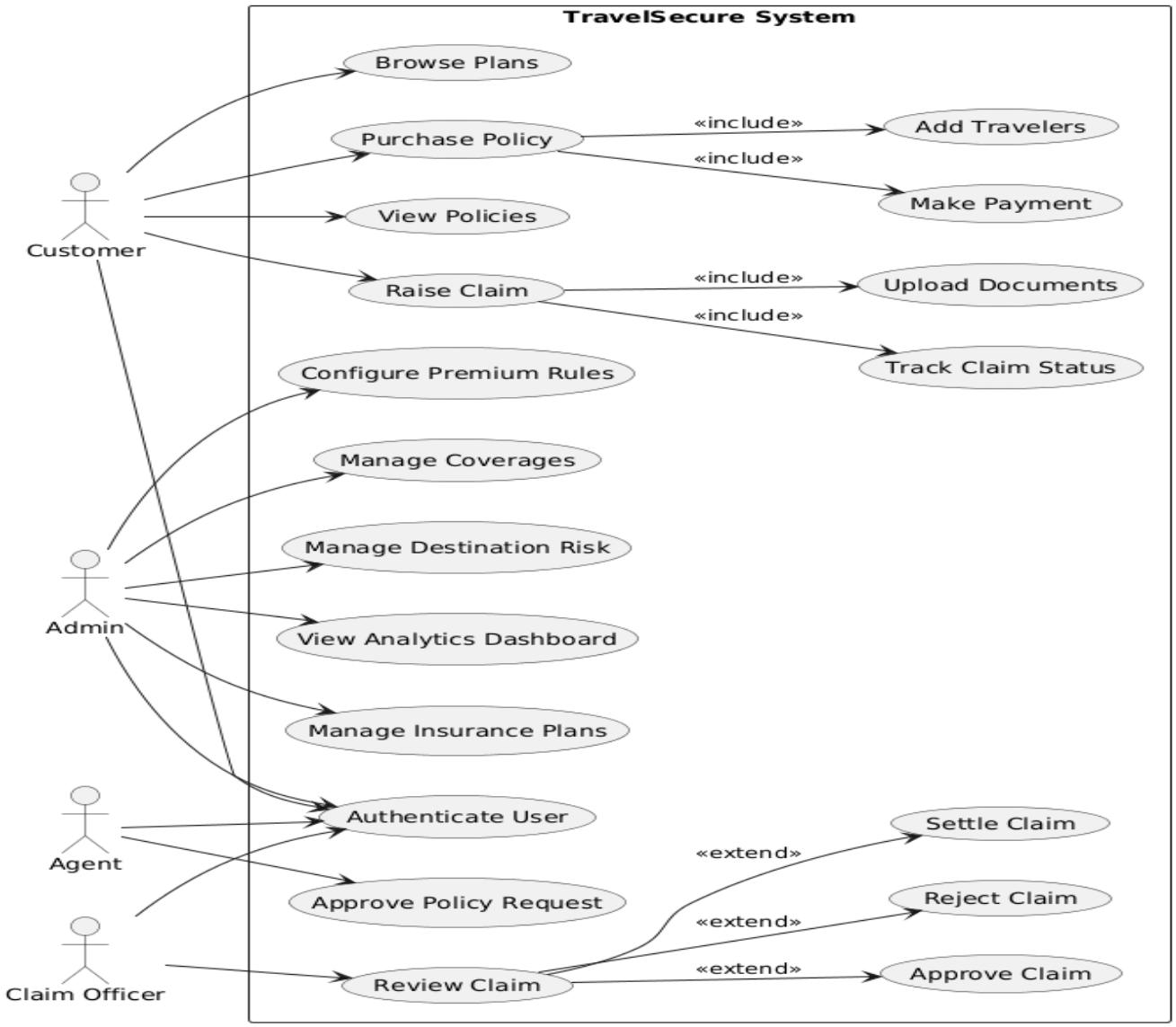


Figure 9.1 – TravelSecure Use Case Diagram

9.2 Entity-Relationship (ER) Diagram

The ER Diagram shows the complete relational schema of the TravelSecure database. It defines all ten entities – Users, InsurancePlans, Coverages, PremiumRules, DestinationRisks, Policies, Travelers, Claims, ClaimDocuments, and Payments – along with their attributes, primary keys, foreign keys, and cardinality relationships.

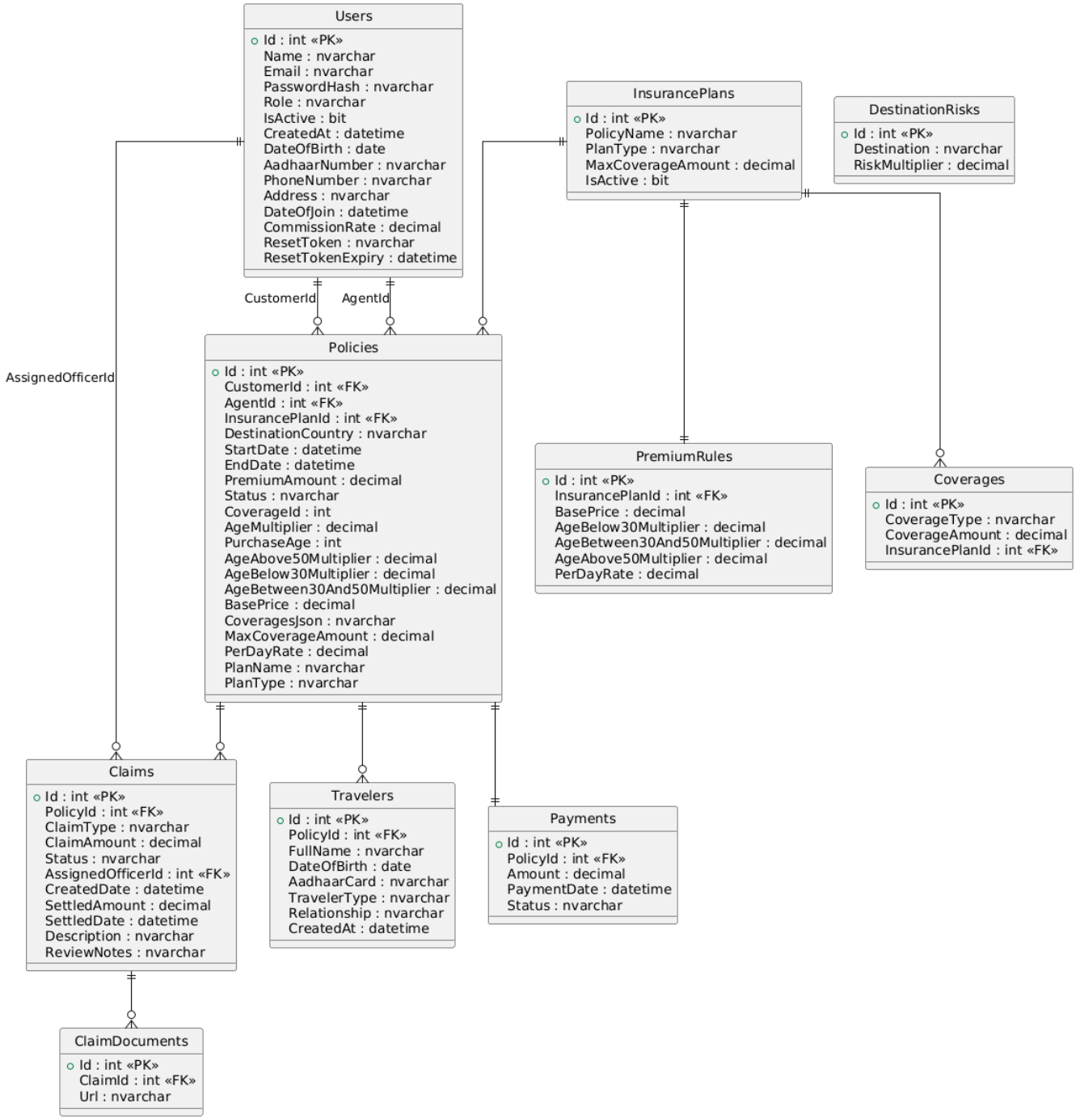


Figure 9.2 – TravelSecure Entity-Relationship Diagram

9.3 Activity Diagram

The Activity Diagram captures the end-to-end workflow of the TravelSecure system across all roles. It models the sequential and parallel activities from initial authentication through Admin system setup, customer policy application, Agent approval, policy purchase and activation, claim submission, and Claim Officer review – reflecting the complete operational lifecycle of the platform.

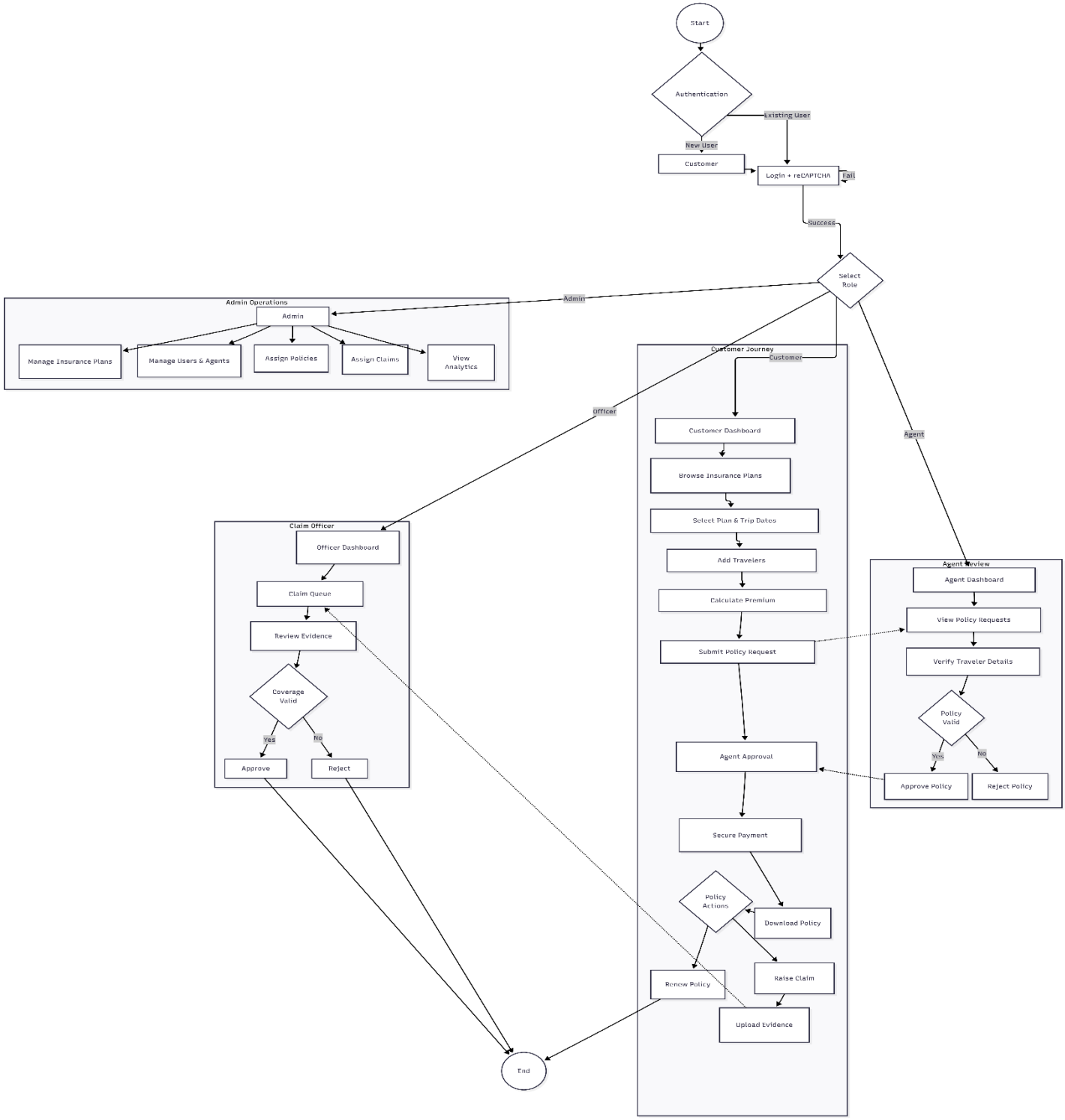
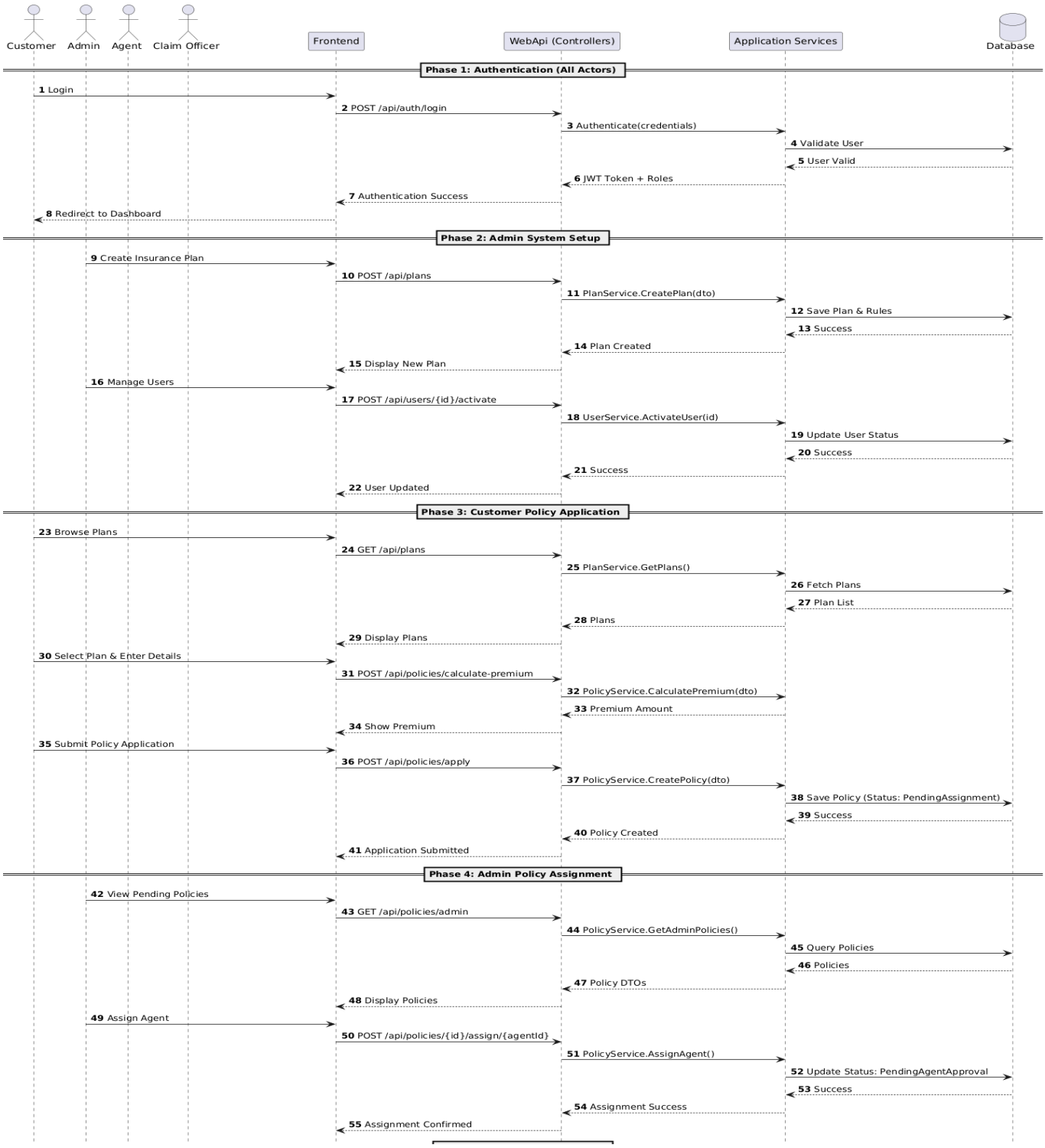


Figure 9.3 – TravelSecure Activity Diagram

9.4 Sequence Diagram

The Sequence Diagram details the chronological message flow between the Frontend, WebApi Controllers, Application Services, and Database layers across eight operational phases: Authentication, Admin System Setup, Customer Policy Application, Admin Policy Assignment, Agent Policy Approval, Policy Purchase and Activation, Claim Submission, and Claim Officer Review.



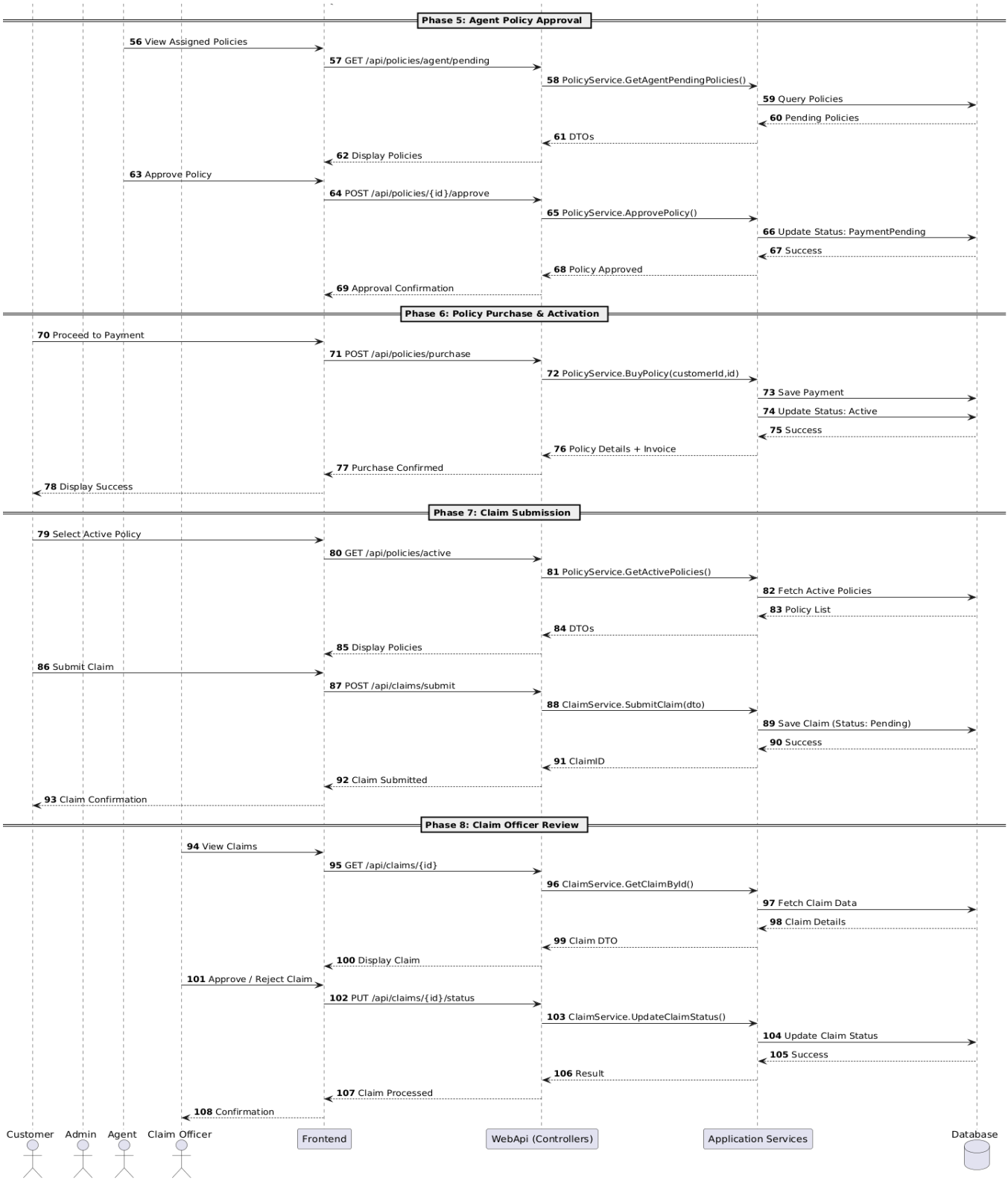


Figure 9.4 – TravelSecure Sequence Diagram

10. Appendices

10.1 Glossary of Domain Terms

Term	Definition
Interested	Policy state where the customer has saved trip details but payment and/or Agent approval is still pending.
PaymentPending	Policy has been approved by an Agent and is awaiting premium payment from the customer.
Active	Premium has been paid and insurance coverage is live for the defined period.
Expired	Policy end date has passed; no further claims can be raised against this policy.
Cancelled	Policy was cancelled by the customer or Administrator within the eligible cancellation window.
Settled	Claim has been manually reviewed and approved by the Claim Officer, and the settlement amount has been recorded.

10.2 Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SPA	Single Page Application
DTO	Data Transfer Object
JWT	JSON Web Token
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper
CORS	Cross-Origin Resource Sharing
API	Application Programming Interface
NFR	Non-Functional Requirement
CI/CD	Continuous Integration / Continuous Deployment
TLS	Transport Layer Security
PII	Personally Identifiable Information

10.3 Policy State Transition Summary

From State	To State	Trigger
(New)	Interested	Customer saves trip details without completing payment.
Interested	PendingAgentApprove	Admin assigns the policy to one of the agent
PendingAgentApprove	PaymentPending	Agent approves the policy request.
PaymentPending	Active	Customer completes premium payment successfully.
Active	Active	Customer renews/extends the policy with additional payment.
Active	Expired	Policy end date is reached with no renewal.
Interested	Cancelled	Customer or Admin cancels within 24-hour window.
PaymentPending	Cancelled	Payment fails or customer withdraws request.